

Lab 1: Flipping an array
Geavanceerde Computerarchitectuur: Labosessie

(B-KUL-JLI1QS)

Timon Coen

February 18, 2024



Group: MEIswe

Instructor: Alex Marinšek

Introduction

In this lab session, the goal is to use the GPU to flip an array. This to illustrate the GPU can make use of computing and executing code in parallel. We are using the CUDA (Compute Unified Device Architecture) model to program NVIDIA GPUs. GPUs exist out of a lot of processing cores, each able to execute instructions concurrently. To program the GPU, we can make use of the CUDA programming model. CUDA provides a high-level programming interface, so the programming is relatively simple.

What's the difference between CPU and GPU programming?

To execute code in parallel, we write functions, called kernels. When executing the kernel, you need to specify the parameters with how many threads you want the code to be executed.
Next to the threads, you can also specify a block size. Within a block, the maximum amount of threads is 1024. So for more than 1024 threads, you need to use multiple blocks.

Flipping the array

For flipping the array, the n-th element will go on place $N - n - 1$. Calculating the n can be done by `n = blockIdx.x * blockDim.x + threadIdx.x`. Where `blockIdx.x` is the index of the blocks, for example the i-th block. That index is multiplied by the dimension of the block: `blockDim.x`. And added to that multiplication is `threadIdx.x`, the index within the block. The `.x` signifies the x dimension. There is a possibility to define 3 dimensions, but in the case of an array, we only need one dimension.

How many threads per block?

The big question is; what is an appropriate block size? Threads within a block are executed simultaneously. But if there are too many blocks to be executed, some of them will need to be executed in sequential order. As discussed before, the maximum block size is 1024 threads, the minimum is one thread. The problem with a block size that is too small, is that each block brings some overhead with it. In this case, if the array is larger than 1024, you can do multiple things. If the number of elements in the array can be fully devised into equal parts below 1024, you can work with block sizes that are fully filled.
In the other case, you can work with an arbitrary number and then limit it and define the threads, but they will be doing no useful work, so they are wasted. In the situation in figure 1, you can see that if you have an array with 1027 elements, and you are using a block size of 1024, you are wasting 1020 threads, so this can be more efficient.
Using a block size of 256 threads, will only waste 252 threads, but 5 blocks will need to be used, which will create more overhead but less wasted threads.
A combination with division can also be used. The block size can be dynamically chosen based on the number of elements in the array, so there are both a small number of threads that are not used and a small number of blocks.

Conclusion

In general, we would like the kernels to be small enough and only execute the code that needs to be executed in parallel. As it would be inefficient to calculate something that is the same for all threads, that could better be done once.
Choosing the right block size to limit both overhead and allocating threads that don't need to do anything.

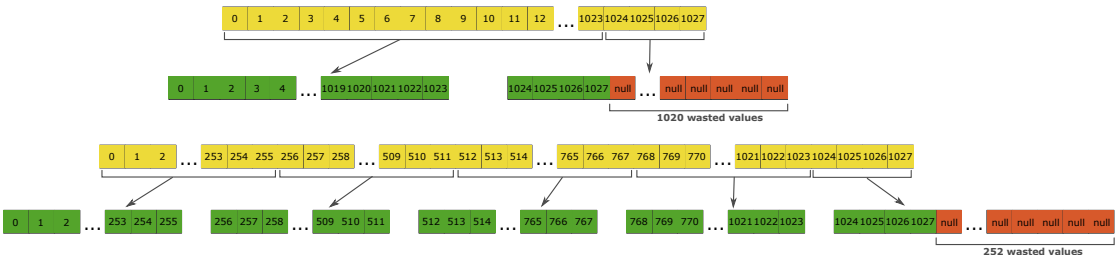


Figure 1: Example of how the block size can have an influence on wasted threads